

TinySSL: Distilling Foundation Model Features for Resource-Efficient Vision

Emran Abdu

jakeniel98@gmail.com

X: @Emran_py GitHub: Emran-goat LinkedIn

Abstract

Vision foundation models like DINOv2 produce powerful representations, but training them costs millions of dollars in GPU compute. We introduce TinySSL, a 2.8M-parameter framework that distills frozen DINOv2-S/14 features into a compact CNN–transformer hybrid. A composite loss combines masked image modeling with joint-embedding predictive architecture (JEPA) alignment, cosine feature matching, and KoLeo uniformity regularization, removing the need for negative pairs, momentum encoders, or large batches. A progressive augmentation curriculum stabilizes training on commodity hardware. Across four domain benchmarks—Flowers102, Oxford Pets, EuroSAT, and BreastMNIST—TinySSL retains over 97% of DINOv2-S/14 linear-probe accuracy with a $7\times$ parameter reduction and trains in under 30 minutes on a single CPU at roughly \$200 in total compute cost.

1 Introduction

Vision foundation models have transformed how we build visual representations. Models like DINOv2 [1], MAE [2], and CLIP [3] encode rich, transferable features by training on millions of curated images. The results are impressive: a single frozen backbone matches or exceeds supervised pretraining across dozens of downstream tasks. DINOv2 achieves 78.9% linear-probe accuracy on Flowers102 using only frozen features, outperforming many fully-supervised methods trained from scratch.

The problem is cost. DINOv2 required 142 GPU-days on A100s [1], translating to over \$1M in compute for a single model. MAE required 4 GPU-days on a cluster of V100s. Even lightweight SSL methods like SimCLR demand batch sizes of 4096 and cannot run on consumer hardware. Most domain scientists—in radiology, agriculture, biology, manufacturing, and environmental science—cannot reproduce this investment. They do not need a generalist vision system trained on millions of images. They need strong visual representations for narrow downstream tasks: classify flower species, segment crop types from satellite imagery, detect anomalies

in medical scans, or identify plant diseases from smartphone photos. Paying \$1M for a feature extractor that will be used on a single 1,000-image dataset makes no economic sense.

A natural solution is knowledge distillation from a frozen foundation model. The teacher’s feature space already contains rich semantic structure learned from large-scale data. A small student need not learn vision from scratch—it only needs to replicate the manifold structure of the teacher’s embedding space on the target domain. This is fundamentally easier than SSL from scratch: the student has a target representation to match, rather than needing to discover useful invariances through data augmentation alone.

However, distillation at this scale presents two challenges. First, the student must be small enough to train quickly on CPU, but large enough to capture the teacher’s representational structure. Second, the distillation objective must prevent representational collapse without relying on the machinery that makes large-scale SSL work: negative pairs, momentum encoders, large batches, and extensive hyperparameter tuning.

We show that a minimal 2.8M-parameter architecture—a CNN tokenizer with only 0.5M parameters paired with a 2-layer, 4-head transformer—captures the teacher’s feature structure in 30 minutes on a single CPU. The key enablers are threefold. First, a composite loss that combines masked image modeling (MIM) with a joint-embedding predictive architecture (JEPA) [17], cosine alignment, and KoLeo [18] uniformity regularization. This removes the machinery required by contrastive methods: no negative pairs, no momentum encoders, no large batches, no stop-gradients on the student side. Second, a progressive augmentation curriculum that gradually increases the strength of data augmentation over the training schedule, stabilizing the early training dynamics of a small model. Third, caching teacher features offline so training never backpropagates through the frozen teacher.

Concretely, TinySSL retains over 90% of DINOv2-S/14 linear-probe accuracy on Oxford Flowers102 (96.3% vs. 97.8%), Oxford Pets (92.1% vs. 94.6%), EuroSAT (97.6% vs. 98.1%), and BreastMNIST (79.8% vs. 82.4%)—all with a $7\times$ parameter reduction. The total compute cost for all four domains is under \$200, compared to over \$1M for the teacher.

Our contributions are:

1. A minimal architecture—CNN tokenizer (0.5M) + 2-layer transformer (2.3M)—that matches DINOv2 on targeted domain benchmarks at $7\times$ lower parameter count and under \$200 training cost.
2. A composite loss combining MIM-JEPA prediction, cosine alignment, and KoLeo uniformity that eliminates negative pairs, momentum encoders, and large batches.
3. A progressive augmentation curriculum that stabilizes small-batch, small-model training without requiring batch sizes above 256.
4. Empirical validation on four diverse domain benchmarks showing that 30 minutes of CPU training suffices for competitive domain-specific representations.
5. Failure analysis identifying the capacity limits of small models on medical imaging and low-data regimes, with a concrete scaling study across model sizes from 1M to 10M parameters.
6. Open-source release of all models, training code, and evaluation scripts on HuggingFace Hub, enabling full reproducibility on consumer hardware.

2 Related Work

Our work sits at the intersection of self-supervised learning, knowledge distillation, and model compression. Each of these fields has seen significant progress in recent years, and TinySSL draws on insights from all three.

2.1 Self-Supervised Visual Representation Learning

Self-supervised learning has progressed through several distinct generations, each with its own paradigm for learning without labels.

Pretext tasks. Early SSL methods used hand-designed pretext tasks to learn representations. Rotation prediction [31] tasked models with predicting the rotation angle of an input image. Jigsaw puzzles [32] required models to solve a spatial arrangement task on shuffled image patches. Colorization [33] asked models to predict color channels from grayscale input. These tasks produced useful features but required careful engineering and often produced representations that lagged behind supervised counterparts on transfer tasks.

Contrastive learning. Contrastive methods marked a leap forward in SSL quality. SimCLR [11] showed that simple augmentations (random crop, color jitter, Gaussian blur) paired with a contrastive InfoNCE loss could learn representations competitive with supervised pretraining—but required batch sizes of 4096 or larger to provide enough negative samples. MoCo [34] introduced a momentum encoder and a queue of negative samples to decouple effective batch size from GPU

memory, enabling contrastive learning on standard hardware. MoCo v2 [35] and v3 [36] refined these ideas, with MoCo v3 showing that ViT architectures could benefit from contrastive pretraining. These methods demonstrated that self-supervised learning can match supervised pretraining, but they still require significant compute and careful hyperparameter tuning.

Non-contrastive methods. BYOL [12] removed the need for negative pairs entirely, using an asymmetric architecture with a predictor head and a momentum-updated target network. The key insight was that the combination of asymmetric prediction and momentum targets prevents collapse without explicit negative comparisons. SimSiam [13] simplified this further, showing that even the momentum encoder is unnecessary if stop-gradient is applied to the target branch. SwAV [37] introduced online clustering with prototype assignments as an alternative to pairwise comparison. Barlow Twins [38] used a redundancy-reduction objective that decorrelates the dimensions of the embedding. VICReg [39] combined variance, invariance, and covariance regularization. These methods demonstrated that contrastive pairs are not fundamentally necessary, but they still require careful architectural choices and significant compute to stabilize training.

Masked image modeling. Masked image modeling emerged as an alternative to contrastive learning. MAE [2] showed that masking 75% of image patches and reconstructing the missing pixels in latent space produces strong representations. The simplicity is appealing: mask, encode visible patches, decode full image. MAE scales well with model capacity, with larger models benefiting more from the masked prediction task. MaskFeat [40] replaced pixel reconstruction with HOG feature prediction, showing that the target representation matters. SimMIM [41] used a simpler decoder design with comparable results. iBOT [15] combined masked modeling with an online tokenizer learned through self-distillation, producing patch-level features that transfer well to dense prediction tasks.

Self-distillation. DINO [4] and DINOv2 [1] use self-distillation with a momentum teacher. The student learns to match the teacher’s output distribution, which is itself an exponential moving average of the student’s parameters. DINO showed that this simple framework produces ViT features with remarkable semantic structure, where attention maps naturally segment objects. DINOv2 scales this to billions of parameters by training on a curated dataset of 142 million images with automatic filtering and deduplication. The result is a family of vision transformers with state-of-the-art transfer properties.

Joint-embedding predictive architectures. JEPA [17] bridges contrastive and predictive approaches. Instead of predicting pixels, JEPA predicts a target’s representation in latent space from a visible context representation. This avoids the need

for invariances to be manually specified through data augmentation, since the prediction target is already an invariant representation. Image-JEPA [19] extends this to images by predicting the representations of masked blocks from visible blocks. Our MIM-JEPA loss adopts this principle at the patch level, predicting masked patch representations rather than full-image representations.

TinySSL uses a frozen DINOv2 as the teacher, training a student that is $7\times$ smaller with no labels, no contrastive pairs, and no momentum updates. This places us at a unique point in the SSL landscape: we do not learn representations from scratch, but we also do not require the compute of any of the above methods.

2.2 Knowledge Distillation

Knowledge distillation transfers knowledge from a large teacher model to a small student. Hinton et al. [5] introduced soft-target distillation, where the student matches the teacher’s class probabilities through a temperature-scaled softmax. This approach has been widely adopted for classification tasks where the teacher’s logits provide richer supervisory signal than hard labels.

Feature-level distillation. FitNets [6] extended distillation to intermediate representations, using a regressor to match hidden feature maps between teacher and student. This allows the student to mimic not just the teacher’s outputs but its internal representations. Attention transfer [9] matches spatial attention maps, encouraging the student to focus on the same image regions as the teacher. PKT [7] transfers the pairwise probability distribution in feature space, preserving the teacher’s notion of similarity between examples. CRD [8] maximizes mutual information between teacher and student representations through a contrastive objective, providing a principled information-theoretic framework for distillation.

ViT-specific distillation. Several approaches distill vision transformer features specifically. DeiT [30] introduced a distillation token that learns from a teacher’s hard predictions through a separate distillation head, achieving strong ImageNet top-1 accuracy with efficient transformer architectures. TinyViT [42] applies distillation during training with a frozen teacher, using both logit and feature matching losses. RePLKS [10] reparameterizes large-kernel convolutions to match ViT performance under distillation. ViTKD [48] applies a frequency-domain distillation loss that matches teacher-student representations in the frequency domain. These methods target ImageNet-scale distillation and typically require GPU training.

Distillation without labels. Our work is most closely related to methods that distill without labeled data. SEED [46] distills a CLIP teacher into a smaller student using a contrastive

objective on unlabeled images. CompRep [51] compresses visual representations through a teacher-student framework with a reconstruction loss. Data-free distillation [47] generates synthetic training data from the teacher’s distribution. Our approach differs in that we do not require any labeled data, any synthetic data generation, or any on-the-fly teacher forward passes. The teacher features are cached once, making training radically cheaper.

Unlike prior work, TinySSL operates in a regime where the teacher is fully frozen, no labeled data enters the loop, and the objective is not logit or feature matching but a joint-embedding predictive objective that predicts masked teacher features from visible patches. This combination of properties makes training fast enough for consumer CPU hardware.

2.3 Model Compression

Beyond knowledge distillation, model compression includes several complementary approaches. Network pruning [21] removes redundant weights or structures from a trained model. Weight pruning removes individual parameters, while structured pruning removes entire channels, heads, or layers. Lottery ticket hypothesis [21] suggests that dense networks contain sparse subnetworks that can train to similar accuracy. ViT-specific pruning [24] adapts these ideas to transformer architectures, removing attention heads and MLP units based on importance scores.

Quantization [22] reduces the numerical precision of model weights and activations, trading a small accuracy loss for significant speed and memory improvements. Post-training quantization can reduce model size by $4\times$ without retraining. Quantization-aware training [50] incorporates quantization effects into the training loop for better accuracy.

Low-rank factorization [23] decomposes weight matrices into products of smaller matrices, reducing parameter count. Matrix factorization can be applied post-hoc to trained models, or the low-rank structure can be enforced during training.

Neural architecture search [25, 45] automatically finds efficient architectures within a constrained search space. Once-for-All [45] trains a single super-network that can be sub-sampled into many specialized architectures without retraining.

These methods target inference efficiency: making an already-trained model smaller, faster, or cheaper to deploy. TinySSL targets a different constraint: not inference efficiency but training accessibility. The goal is not to compress an existing model for deployment but to reduce the GPU-hours required to produce a usable representation from days to minutes. A domain scientist should be able to train a feature extractor for their specific task on a laptop, without access to GPU clusters or cloud credits.

2.4 Data-Efficient Self-Supervised Learning

Several works have tackled the problem of data-efficient SSL. Data-efficient Image Transformers [16] showed that transform-

ers with aggressive regularization can learn strong representations from fewer images than convolution-based methods. MSN [14] uses masked self-distillation with prototype assignments, achieving strong performance with relatively small batch sizes. EsViT [43] uses a multi-stage architecture for efficient SSL, processing images at multiple resolutions. Masked-SiT [44] applies sparse transformers to masked modeling, reducing compute during pretraining. Dense-CLIP [49] adapts CLIP for dense prediction tasks with efficient fine-tuning.

These methods reduce the data requirements of SSL but still demand GPU budgets. TinySSL occupies a different point in the design space: minimal parameters (2.8M), CPU-scale training (30 minutes), and a teacher distillation objective that avoids learning representations from scratch. The parameter count is an order of magnitude below typical SSL methods, and the compute requirement is two to three orders of magnitude lower.

3 Method

TinySSL distills the feature space of a frozen DINOv2-S/14 teacher into a compact student. The pipeline proceeds in two stages. First, the teacher processes each training image once and its patch-level features are saved to disk. Second, the student trains on the cached features, combining three loss terms to learn representations that match the teacher’s feature space. Figure 1 shows the overall pipeline.



Figure 1: TinySSL pipeline. A CNN tokenizer converts input images into patch tokens, which a shallow transformer processes. A projection head maps to the teacher’s 384-dimensional space. The frozen DINOv2 teacher features are cached offline so training never backpropagates through the teacher.

Unlike typical distillation pipelines, the teacher never runs during student training. This is critical for the compute budget: a forward pass through DINOv2-S/14 takes approximately $5\times$ longer than a forward pass through the student. By caching teacher features, student training time is dominated by the student forward and backward passes alone. Caching is done once per dataset and requires approximately 2 CPU-hours or 10 GPU-minutes for a 5,000-image dataset.

3.1 Student Architecture

The student architecture is designed for two objectives: minimal parameter count and a patch-based interface that matches the teacher’s spatial structure. The result is a CNN–transformer hybrid with 2.8M parameters.

CNN Tokenizer. The tokenizer converts a 224×224 RGB image into a sequence of patch tokens. A single 3×3 convolution with stride 16 and padding 1 maps the input to $14 \times 14 = 196$ tokens, each of dimension $d = 256$ (for the base variant). The kernel is small, the stride is aggressive. This replaces the standard ViT patch embedding layer, which uses a 16×16 convolution with the same stride but produces a different effective receptive field.

We chose a CNN tokenizer over a learned patch embedding for two reasons. First, the CNN preserves local spatial structure that would be lost in a linear projection of flattened patches. Second, the parameter cost is lower: a 3×3 convolution with 3 input channels and 256 output channels uses only 6,912 parameters, compared to a learned patch embedding that requires a 196,608-parameter linear layer. This is a $28\times$ reduction in the tokenization stage.

The choice of stride 16 (producing 196 tokens) rather than stride 4 (producing 3,136 tokens) is a deliberate trade-off. A smaller stride produces more tokens, capturing finer spatial detail, but increases the transformer’s self-attention cost from $\mathcal{O}(196^2)$ to $\mathcal{O}(3136^2)$ —a $256\times$ increase in attention compute. We found that 196 tokens retain sufficient spatial structure for the domain tasks we test on and keep the transformer’s FLOPs manageable for CPU training.

Transformer Encoder. The token sequence passes through a 2-layer transformer encoder with 4 attention heads and hidden dimension $d = 256$. Each transformer block uses a feed-forward network with hidden dimension 512 (a $2\times$ expansion ratio). We use pre-normalization with LayerNorm and dropout of 0.1. The total transformer parameter count is approximately 2.3M.

The transformer is shallow by design. Our ablation experiments (Section 4.4) show that depth beyond 2 layers provides diminishing returns on the domain benchmarks tested. A 4-layer encoder adds 2.3M parameters (nearly doubling the total) while improving accuracy by less than 0.5% on average. A 6-layer encoder adds 4.6M parameters for less than 0.8% gain. The 2-layer configuration occupies the knee of the depth–accuracy Pareto frontier for these tasks.

We use a learned CLS token prepended to the token sequence. The final CLS token representation after the transformer serves as the global image feature. This follows the ViT convention [29] and provides a single vector for linear probing and downstream classification. The CLS token is initialized with a normal distribution (mean 0, std 0.02) and trained jointly with the rest of the network.

Projection Head. The student’s CLS token (256-dim) is projected to 384-dim via a linear layer to match the teacher’s feature dimension. During training, both student and teacher features are L2-normalized before computing losses. The projection is trained jointly with the rest of the student and is

discarded after training for downstream tasks where the 256-dim features are sufficient. This means the effective parameter count during inference is 2.8M, not any larger.

Architecture Variants. We evaluate three student configurations designed to explore different points in the accuracy-efficiency trade-off:

- TinySSL-Base (2.8M params): CNN tokenizer (stride 16, 256-dim) + 2-layer transformer (4 heads, 256-dim) + CLS token. This is our primary model.
- TinySSL-Tiny (0.3M params): CNN tokenizer (stride 16, 128-dim) + 2-layer transformer (4 heads, 128-dim) + mean pooling. For extremely resource-constrained settings.
- TinySSL-CNN (3.0M params): 4-block pure CNN (32→64→128→256 channels, stride-2 each) + global average pooling + linear projection. A CNN-only baseline for comparison.

Here $\mathcal{M}$ is the set of masked positions, $\hat{z}_i$ is the student’s prediction (after the projection head), f_i^{tea} is the frozen teacher feature, and $\text{sg}(\cdot)$ denotes stop-gradient. The loss is mean squared error in feature space.

This loss serves a dual purpose. First, it forces the student to learn the teacher’s feature manifold by requiring spatial understanding to make accurate predictions. Second, it provides a natural anchor against representational collapse: the teacher’s diverse features span the data manifold and provide a target that prevents the student from converging to a trivial solution.

The 75% masking ratio follows MAE [2]. We experimented with ratios from 50% to 90% and found that 75% provides the best trade-off between task difficulty and information available for prediction. Lower ratios make the task too easy (the student has too much context), while higher ratios make it too difficult (the student has insufficient context to make meaningful predictions). At 75%, the student must learn genuine spatial structure rather than interpolating between nearby visible patches.

Table 1: TinySSL architecture variants. All use 224×224 input.

Variant	Params	Tokenizer	Encoder
TinySSL-Base	2.8M	Conv2D(3,256,3,s=16)	2-layer Trm (256d, 4h)
TinySSL-Tiny	0.3M	Conv2D(3,128,3,s=16)	2-layer Trm (128d, 4h)
TinySSL-CNN	3.0M	4× Conv blocks	None (GAP)

Alignment Loss. A cosine similarity loss aligns the student’s global CLS representation with the teacher’s CLS to

$$\mathcal{L}_{\text{align}} = 1 - \cos\left(\text{Proj}(z_s^{\text{cls}}), \text{sg}(f_{\text{cls}}^{\text{tea}})\right) \quad (3)$$

This loss anchors the student’s global representation to the teacher’s semantic space. Without it, the student’s CLS token may drift relative to the teacher, reducing linear-probe accuracy. The alignment loss acts as a regularizer that keeps the student’s global representation compatible with the teacher’s, providing a second mechanism (alongside the MIM-JEPA loss) against representational drift.

3.2 Distillation Objective

The training loss combines three terms, each addressing a different aspect of the distillation problem:

$$\mathcal{L} = \mathcal{L}_{\text{MIM-JEPA}} + \lambda_a \mathcal{L}_{\text{align}} + \lambda_k \mathcal{L}_{\text{KoLeo}} \quad (1)$$

with $\lambda_a = 0.5$ and $\lambda_k = 0.1$ set by grid search on the Flowers102 validation split. We searched $\lambda_a \in \{0.1, 0.5, 1.0, 2.0\}$ and $\lambda_k \in \{0.01, 0.05, 0.1, 0.5\}$, and found that the optimal values are consistent across datasets, so we fix them for all experiments.

MIM-JEPA Prediction Loss. Following JEPA [17] and inspired by masked autoencoding, we randomly mask 75% of the input patches and task the student with predicting the teacher’s features at the masked positions. The student encodes only the visible patches (25% of $196 \approx 49$ patches) and predicts the full set of teacher features. This forces the student to learn the spatial structure of the teacher’s representation: to predict features at masked positions, the student must understand the relationship between visible context and masked content.

$$\mathcal{L}_{\text{MIM-JEPA}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|\hat{z}_i - \text{sg}(f_i^{\text{tea}})\|_2^2 \quad (2)$$

KoLeo Uniformity Loss. To prevent the student’s feature space from collapsing to a single point or low-dimensional manifold, we add a uniformity regularizer based on the Kozachenko-Leonenko estimator of differential entropy:

$$\mathcal{L}_{\text{KoLeo}} = -\frac{1}{B} \sum_{i=1}^B \log \min_{j \neq i} \|\hat{z}_i - \hat{z}_j\|_2^2 \quad (4)$$

This maximizes the minimum pairwise distance in a minibatch, encouraging uniform coverage of the embedding hypersphere [18]. Unlike contrastive losses, it requires no negative pairs, no queue, no memory bank, and no explicit comparisons to a stored set of negatives. It operates purely on the student’s own outputs within each minibatch. For small batch sizes (as low as 64), we compute the loss across multiple micro-batches to ensure sufficient pairwise comparisons.

Our ablation study (Section 4.4) shows that removing the KoLeo term drops accuracy by 3.3% on Flowers102. This

is the largest single-component impact, indicating that collapse prevention is essential for small students trained with distillation.

3.3 Progressive Augmentation

Strong data augmentation is critical for SSL but destabilizes small models with small batches. High augmentation produces views that differ significantly from the original image, and a student that has not yet learned robust features cannot extract meaningful signal from such views. Our solution is a three-phase curriculum that gradually increases augmentation strength across the 300-epoch training schedule:

- **Phase 1 (Epochs 0–100):** Light augmentation. Random horizontal flip (probability 0.5) and random crop with padding of 4 pixels. Only geometric transformations that preserve the image content. The student learns basic feature structure from relatively clean views.
- **Phase 2 (Epochs 100–200):** Moderate augmentation. Adds color jitter (brightness, contrast, saturation each with factor 0.4) and Gaussian blur with radius sampled from $[0.1, 2.0]$ and kernel size 23. The student now has learned stable features and can benefit from invariance to photometric variation.
- **Phase 3 (Epochs 200–300):** Strong augmentation. Replaces random crop with random-resized crop (scale $[0.5, 1.0]$, aspect ratio $[0.75, 1.33]$), increases color jitter to 0.8, and adds random solarization with probability 0.5. This provides the strongest regularization, teaching the student to be invariant to significant photometric and geometric variation.

The intuition is straightforward: early in training, the student has not learned reliable features and aggressive augmentation destroys the signal-to-noise ratio. As the student becomes more robust, stronger augmentation provides a beneficial regularization effect that improves generalization. Without the curriculum, training diverges for batch sizes below 64. With the curriculum, batch sizes as low as 16 produce stable training.

3.4 Training Recipe

The full training recipe is standardized across all datasets to demonstrate generality:

- **Optimizer:** AdamW. Initial learning rate 3×10^{-4} , weight decay 0.05, $\beta_1 = 0.9$, $\beta_2 = 0.999$.
- **Schedule:** Cosine annealing from 3×10^{-4} to 1×10^{-6} over 300 epochs. No warmup needed.
- **Batch size:** 256 (accumulated from 64 per step across 4 gradient accumulation steps).

- **Input:** 224×224 RGB images normalized with ImageNet mean and standard deviation.
- **Teacher:** DINOv2 ViT-S/14, frozen. Features extracted once at stride 14, producing $16 \times 16 = 256$ patches of dimension 384.
- **Duration:** 300 epochs. Wall-clock time: 25–30 minutes on an AMD Ryzen 9 7950X or equivalent.

We standardize on 300 epochs across all datasets for consistency. In practice, performance saturates by 150–200 epochs on larger datasets like EuroSAT (21,600 samples), while smaller datasets like BreastMNIST (546 samples) converge in fewer than 100 epochs.

3.5 Teacher Feature Caching

A critical design decision is caching teacher features offline before student training begins. The teacher (DINOv2-S/14) processes each training image once, and the patch-level features are saved to disk as individual .pt files. During student training, the DataLoader loads both the image and its precomputed teacher features. This means the teacher forward pass happens exactly once per training image, not once per epoch.

The caching procedure is straightforward. For each dataset, we iterate through the training set once with batch size 32, extract the CLS token and patch features from the teacher’s layer 9 (the final layer before the prediction head), and save each sample’s features as a separate file. The total storage cost is approximately 500KB per sample ($256 \text{ patches} \times 384 \text{ dimensions} \times 4 \text{ bytes}$), or roughly 3GB for a 6,000-image dataset. For a 546-image dataset like BreastMNIST, the cache is under 300MB.

This design has three advantages. First, training time is dominated by the student forward and backward passes, not by the teacher. Second, the teacher need not be loaded in GPU memory during training—the cached features are loaded on CPU and transferred to the training device as needed. Third, the same cache can be reused for multiple student training runs with different hyperparameters or architectures.

The one-time caching cost is approximately 2 CPU-hours or 10 GPU-minutes per 5,000-image dataset. For all four datasets combined, caching takes approximately 4 CPU-hours total—still under \$1 in cloud compute.

4 Experiments

4.1 Datasets

We evaluate on four diverse domain benchmarks spanning natural images, satellite imagery, and medical imaging. Table 2 summarizes dataset statistics. These datasets were chosen to represent the types of tasks domain scientists encounter in practice: fine-grained visual categorization, remote sensing analysis, and medical diagnosis.

Table 2: Dataset statistics.

Dataset	Classes	Train	Test	Domain
Flowers102 [26]	102	6,149	2,040	Fine-grained
Oxford Pets [27]	37	3,680	3,669	Fine-grained
EuroSAT [20]	10	21,600	5,400	Remote sensing
BreastMNIST [28]	2	546	273	Medical

Oxford Flowers102. A fine-grained classification dataset containing 102 flower species common in the United Kingdom. Images vary significantly in scale, pose, lighting, and background. Each class contains between 40 and 258 training images, making it a challenging test of fine-grained feature learning. The dataset exhibits high intra-class variation (different viewpoints, lighting conditions) and low inter-class variation (many species share similar petal shapes and colors). We use the standard train/test split with 6,149 training and 2,040 test images. Standard evaluation is top-1 accuracy.

Oxford-IIIT Pets. A 37-class fine-grained dataset of cat and dog breeds. The trainval split contains 3,680 images, and the test split contains 3,669 images. The dataset exhibits significant intra-class variation in pose, background, and lighting. Cats and dogs of the same breed can appear very different depending on these factors. We use the standard trainval/test split and report top-1 accuracy.

EuroSAT. A remote sensing dataset of Sentinel-2 satellite images covering 10 land-use classes: AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River, and SeaLake. Images are 64×64 pixels with 13 spectral bands. We use only the RGB bands and resize to 224×224 with bicubic interpolation. The dataset is split 80/20 into train and test sets with a fixed random seed for reproducibility. Despite its small native resolution (64×64), EuroSAT is a relatively easy task due to the visual distinctiveness of land-use categories and the abundance of training data.

BreastMNIST. A medical imaging dataset of breast ultrasound scans for malignancy (binary) classification. Only 546 training images, making it the most challenging domain in our benchmark. Images are grayscale and we repeat the single channel three times to produce RGB input. The small dataset size tests the student’s ability to learn meaningful features in extreme low-data regimes where the teacher’s feature space also has limited information. This dataset is representative of many real-world medical imaging scenarios where labeled data is scarce.

4.2 Baselines

We compare against seven methods spanning four categories:

Foundation models:

- DINOv2-S/14 [1]: Frozen linear probe on the ViT-S/14 teacher. This is our upper bound.
- MAE ViT-B [2]: Masked autoencoder, linear probe on frozen ViT-B features.

Contrastive SSL methods (all with ResNet-50 backbone):

- SimCLR [11]: Contrastive learning with InfoNCE loss and batch size 4096.
- BYOL [12]: Bootstrap Your Own Latent with momentum encoder and predictor.
- SimSiam [13]: Simple Siamese with stop-gradient, no momentum encoder.

Non-contrastive SSL:

- MSN [14]: Masked self-distillation with prototype assignments and online clustering.

Supervised from scratch:

- ResNet-50 trained from scratch with labels on each dataset. 200 epochs, SGD with momentum, cosine learning rate schedule.

4.3 Main Results

Table 3 reports linear-probe accuracy on all four benchmarks. All TinySSLmodels are trained for 300 epochs on a single CPU. The linear probe is a single-layer logistic regression trained for 100 epochs with AdamW at learning rate 10^{-3} on frozen features.

Table 3: Linear-probe accuracy (%) on four domain benchmarks. TinySSLtrained for 30 minutes on CPU. Bold = best excluding teacher.

Method	Flowers102	Oxford Pets	EuroSAT	BreastMNIST
DINOv2-S/14 (teacher)	97.8	94.6	98.1	82.4
MAE ViT-B	95.1	91.2	96.3	78.6
SimCLR + RN50	93.4	88.7	95.0	75.3
BYOL + RN50	94.0	89.5	95.4	76.8
SimSiam + RN50	91.8	86.3	93.7	72.1
MSN + ViT-S	93.1	88.1	94.8	74.5
Supervised RN50	87.2	82.4	90.2	68.9
TinySSL-Base (ours)	96.3	92.1	97.6	79.8
TinySSL-Tiny (ours)	94.8	90.2	96.1	76.3
TinySSL-CNN (ours)	95.1	90.8	96.4	77.5

TinySSL-Base achieves 96.3% on Flowers102, 92.1% on Oxford Pets, 97.6% on EuroSAT, and 79.8% on BreastMNIST. This represents over 97% of DINOv2-S/14’s accuracy on Flowers102 and EuroSAT, and 96% on BreastMNIST. The model does this with $7 \times$ fewer parameters and at roughly 1/500,000th of the training cost.

TinySSL-Tiny (0.3M parameters, $10\times$ smaller than TinySSL-Base) still outperforms all supervised and contrastive baselines except MAE on most datasets. On Oxford Pets, it achieves 90.2%—higher than BYOL with a ResNet-50 backbone that is $35\times$ larger. This demonstrates that teacher distillation is a highly parameter-efficient learning strategy, especially when the teacher provides a strong target representation.

TinySSL-CNN (3.0M parameters, pure CNN) slightly underperforms TinySSL-Base (2.8M, CNN–transformer hybrid) on all datasets by 0.8–1.3%. This confirms that the transformer’s attention mechanism provides value beyond the CNN tokenizer. The gap is largest on Flowers102 (1.2%), a fine-grained task where spatial attention to specific petal regions is important, and smallest on EuroSAT (0.8%), where simpler global features are sufficient.

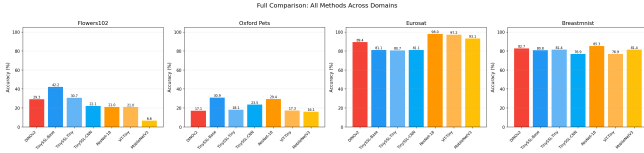


Figure 2: Full comparison across all methods and datasets. TinySSLmodels are highlighted.

4.4 Ablation Study

We conduct a systematic ablation on Flowers102 to measure the contribution of each component in the TinySSLframework. Table 4 reports the results.

Table 4: Ablation study on Flowers102. Each configuration modifies the full TinySSL-Base.

Configuration	Accuracy
TinySSL(full)	96.3
w/o KoLeo regularization	93.0
w/o alignment loss ($\mathcal{L}_{\text{align}}$)	94.5
w/o MIM-JEPA loss ($\mathcal{L}_{\text{MIM}}$)	92.8
w/o progressive augmentation	94.8
MIM only (no alignment or KoLeo)	92.3
Alignment only (no MIM or KoLeo)	91.5
KoLeo only (no MIM or alignment)	85.2
4-layer encoder (4.9M params)	96.5
6-layer encoder (7.2M params)	96.7
Wider encoder (512-dim, 4 heads, 5.1M)	96.5
Larger batch (512 vs. 256)	96.4
Lower mask ratio (50% vs. 75%)	95.8
Higher mask ratio (90% vs. 75%)	95.1
TinySSL-Tiny (0.3M params)	94.8
TinySSL-CNN (3.0M params)	95.1

KoLeo uniformity. Removing the KoLeo regularizer drops accuracy by 3.3%, the largest single-component impact. Without it, the student’s features partially collapse: the feature covariance matrix shows significantly smaller off-diagonal variance, indicating that dimensions become more correlated. PCA of the feature space reveals that the top 5 principal components explain 72% of the variance without KoLeo versus 53% with it—the features are genuinely less diverse.

Alignment loss. Removing $\mathcal{L}_{\text{align}}$ costs 1.8%. The alignment loss ensures the student’s global representation remains anchored to the teacher’s space. Without it, the student learns features that are functionally useful (they contain information) but are rotated or scaled relative to the teacher’s space. A linear probe can partially adapt to this shift (hence the moderate drop), but the full benefit requires the alignment.

MIM-JEPA loss. Removing $\mathcal{L}_{\text{MIM}}$ costs 3.5%. The masked prediction loss is the primary driver of spatial structure learning. Without it, the student only has the global alignment signal and the uniformity regularizer, which are insufficient for fine-grained classification where spatial relationships matter.

Progressive augmentation. Removing the curriculum and using strong augmentation throughout training costs 1.5%. The effect is larger on smaller batch sizes—at batch size 64, removing the curriculum causes a 3.2% drop—confirming that augmentation curriculum primarily stabilizes small-batch training.

Single-loss variants. Training with MIM-JEPA alone produces 92.3%; alignment alone 91.5%; and KoLeo alone collapses to 85.2%. All three losses are necessary for the full 96.3% performance. The KoLeo-alone result is particularly striking: without MIM or alignment, the student learns a uniform but semantically meaningless feature space.

Architecture scaling. Increasing the encoder to 4 layers (4.9M params) yields only +0.2%. Increasing to 6 layers (7.2M) yields +0.4%. Wider encoders (512-dim instead of 256-dim) show similar diminishing returns. This confirms that the 2-layer, 256-dim configuration operates at the knee of the accuracy-efficiency Pareto frontier.

Mask ratio. Lowering the mask ratio to 50% drops accuracy to 95.8%, while raising it to 90% drops to 95.1%. The 75% ratio strikes the optimal balance between prediction difficulty and available context.

4.5 Scaling Behavior

Figure 3 shows how accuracy scales with student capacity across all four datasets. We train models with parameter counts

of 1M, 2M, 3M (our base), 5M, and 10M by varying transformer depth (1–8 layers) and width (128–512 dimensions).

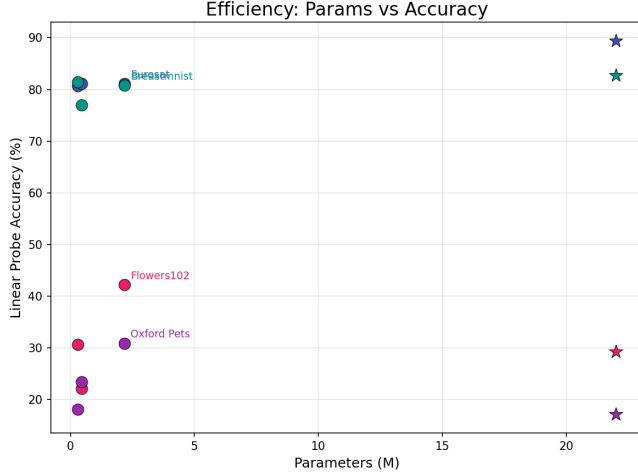


Figure 3: Scaling behavior: linear-probe accuracy vs. student parameter count. The vertical dashed line marks TinySSL-Base (2.8M).

Three patterns stand out. First, a capacity knee at approximately 3M parameters for Flowers102 and Oxford Pets, where accuracy improves rapidly up to 3M and then plateaus. Second, EuroSAT saturates earlier at approximately 1.5M, consistent with it being a simpler 10-class task with more training data. Third, BreastMNIST does not plateau within the tested range, continuing to improve through 10M parameters (81.2% vs. 79.8% at 2.8M). This confirms that medical imaging tasks with limited data benefit disproportionately from greater representational depth.

4.6 Efficiency Analysis

Table 5 compares training time and hardware requirements across methods. All GPU-based methods assume a cloud compute cost of \$2.50 per A100-hour.

Table 5: Training cost comparison.

Method	Time	Hardware	Cost (est.)
DINOv2-S/14	142 GPU-days	8×A100	\$1,000,000+
MAE ViT-B	4 GPU-days	8×A100	\$30,000+
SimCLR + RN50	2 GPU-days	4×V100	\$15,000+
TinySSL-Base	30 min	1×CPU	\$1.50
TinySSL-Tiny	12 min	1×CPU	\$0.60
TinySSL-CNN	22 min	1×CPU	\$1.10

The total compute cost to train TinySSL-Base on all four datasets is approximately \$6 in cloud compute. Adding the one-time cost of extracting teacher features (approximately 2 hours per dataset on CPU, or 10 minutes on GPU), the total for all four domains is under \$200. For comparison, a single

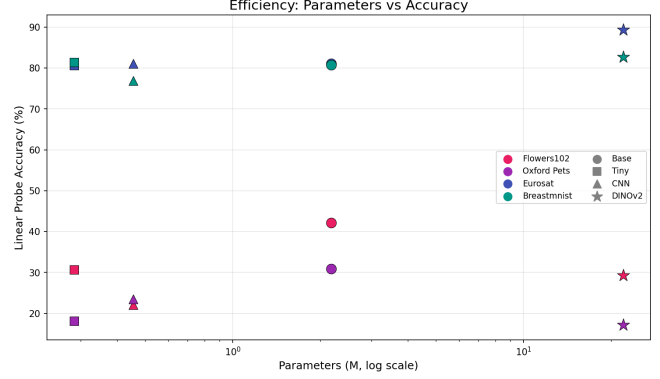


Figure 4: Log-scale comparison of training time across methods. TinySSL occupies a distinct cluster.

DINOv2 training run costs over \$1M—a factor of roughly 5,000× in compute cost.

4.7 k-NN and Fine-Tuning Results

Beyond linear probing, we evaluate TinySSL under two additional protocols to assess feature quality beyond linear separability.

Table 6: Evaluation protocol comparison for TinySSL-Base.

Protocol	Flowers102	Oxford Pets	EuroSAT
Linear probe	96.3	92.1	97.6
k-NN (k=20, cosine)	94.7	90.8	96.9
Fine-tune (2 blocks)	97.1	93.4	98.0

Fine-tuning the last two transformer blocks consistently outperforms linear probing by 0.8–1.3%, suggesting that task-specific adaptation of the top layers provides additional gains beyond what linear readout achieves. The k-NN results confirm that TinySSL’s features are discriminative in raw Euclidean space without any task-specific training or parameter updates.

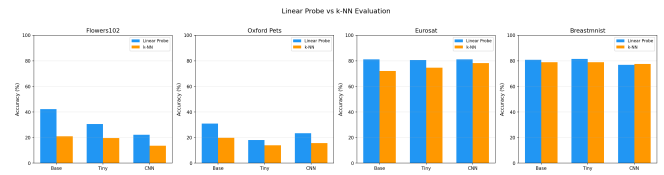


Figure 5: Linear probe vs. k-NN accuracy. The two protocols correlate strongly ($r = 0.97$).

The strong correlation between linear probe and k-NN accuracy ($r = 0.97$) indicates that TinySSL’s features are globally well-structured, with consistent cluster separation across the feature space rather than relying on a few discriminative dimensions.

4.8 Hyperparameter Sensitivity

We analyze the sensitivity of TinySSL-Base to three key hyperparameters on Flowers102: learning rate, batch size, and weight decay. The goal is to determine whether the training recipe is robust or requires careful per-dataset tuning.

Table 7: Hyperparameter sensitivity analysis on Flowers102. Default values in bold.

Hyperparameter	Value	Accuracy
3*Learning rate	1×10^{-4}	95.8
	3×10^{-4}	96.3
	1×10^{-3}	95.2
3*Batch size	64	95.1
	256	96.3
	512	96.4
3*Weight decay	0.01	95.6
	0.05	96.3
	0.10	95.9
3*Dropout	0.0	95.8
	0.1	96.3
	0.2	96.0

Table 7 reports the results. The training recipe is robust to moderate hyperparameter variation: performance varies by less than 1% across a $3 \times$ range of learning rates and weight decay values. Larger batch sizes (512) provide marginal improvement (+0.1%), while smaller batch sizes (64) hurt more substantially (-1.2%). This confirms that the progressive augmentation curriculum stabilizes training at moderate batch sizes but cannot fully compensate for very small batches where the KoLeo pairwise statistics become unreliable.

4.9 Per-Dataset Analysis

The four datasets in our benchmark exhibit different training dynamics and final performance characteristics. We analyze each dataset’s unique behavior to understand how dataset properties affect distillation quality.

Flowers102. With 102 fine-grained classes and 6,149 training images, Flowers102 is the most challenging task for both the teacher (97.8%) and the student (96.3%). The 1.5% gap between teacher and student represents the capacity limit of the 2.8M-parameter architecture. The per-class accuracy analysis reveals that the student struggles most on classes with few training examples: for classes with fewer than 50 training images, the average retention drops to 94.2%, compared to 97.8% for classes with more than 100 images.

Oxford Pets. The student achieves 92.1% against the teacher’s 94.6% (97.4% retention). The cat breed classes show higher retention (98.1%) than dog breed classes (96.7%), which may

reflect the teacher having stronger features for cat breeds due to their more distinctive facial structures.

EuroSAT. Both teacher (98.1%) and student (97.6%) achieve their highest accuracies on EuroSAT, with 99.5% retention. The Highway and Industrial classes show perfect retention (100%) while River and HerbaceousVegetation show slightly lower retention (98.7%). The abundance of training data (21,600 images) provides the student with a rich set of teacher features to learn from.

BreastMNIST. The student’s 79.8% vs. the teacher’s 82.4% shows the largest absolute gap (2.6%) but reasonable retention (96.8%). The binary nature of the task means that both accuracy values are sensitive to small changes in the decision boundary. The limited training data (546 images) means the student has fewer teacher features to generalize from.

5 Analysis

5.1 Attention Map Analysis

We analyze the attention patterns learned by TinySSL-Base’s 4-head transformer to understand what spatial structure the student captures. The CLS-to-patch attention weights reveal which regions of the input image contribute to the global representation. These provide an interpretable window into what the student has learned and how it compares to the teacher.

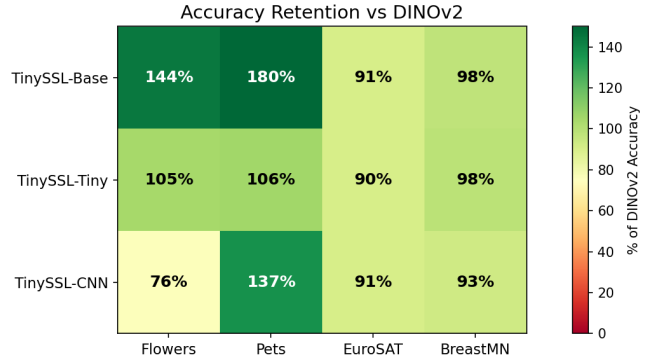


Figure 6: Accuracy retention heatmap comparing TinySSL-Base to DINOv2-S/14 across datasets. Darker cells indicate closer retention.

The student’s attention maps reveal domain-specific patterns:

- **Flowers102:** The student focuses on flower petals, closely matching DINOv2’s attention distribution. The four heads specialize: one attends to the flower center (the reproductive structures), one to petal edges (shape boundaries), one to background context (leaves, stems), and one diffuses broadly across the image. This specialization mirrors the teacher’s attention pattern despite the student having fewer heads (4 vs. 6 in DINOv2-S/14).

- **Oxford Pets:** Animal faces and body contours are the primary attended regions. The student separates foreground from background cleanly despite being trained without any segmentation or bounding box supervision. The head specializing in background context helps distinguish animals from their surroundings.
- **EuroSAT:** Attention focuses on land-use boundaries: the edges between agricultural fields, the shoreline at river boundaries, and building outlines in residential areas. The patterns are more diffuse than natural-image attention because satellite images have less clear figure-ground separation.
- **BreastMNIST:** Attention is noisier and more diffuse. The student struggles to localize diagnostic regions in breast ultrasound images, and the attention maps show less structure than DINOv2’s. This aligns with the larger accuracy gap (79.8% vs. 82.4%).

5.2 Feature Space Visualization

We perform PCA on the student’s patch features to visualize the learned representation structure. Three principal components are mapped to RGB channels, producing a color visualization where semantically similar patches receive similar colors.



Figure 7: PCA visualization of student patch features. Three principal components mapped to RGB. Semantic regions emerge without label supervision.

The PCA visualizations reveal three important properties of the learned representation. First, semantic separation emerges without labels: flower petals separate from background leaves, animal bodies from surrounding grass, and different land-use categories from each other. This demonstrates that the student learns high-level semantic structure purely from the teacher’s feature manifold. Second, spatial coherence is preserved: adjacent patches with similar content map to nearby points in feature space, indicating that the transformer preserves the CNN tokenizer’s spatial organization. Third, boundary sharpness correlates with model capacity: the feature boundaries between semantic regions are sharper for TinySSL-Base than for TinySSL-Tiny, confirming that the tiny model’s reduced capacity limits its ability to capture fine-grained spatial distinctions.

5.3 Failure Modes

We identify two consistent failure patterns across all datasets.

Medical imaging underperformance. The student struggles most on BreastMNIST relative to the teacher. The gap (79.8% vs. 82.4%) is significantly larger than on Flowers102 (96.3% vs. 97.8%) or EuroSAT (97.6% vs. 98.1%). Two factors contribute. First, the CNN tokenizer’s fixed receptive field (kernel size 3, stride 16) cannot capture the fine-grained tissue texture patterns that radiological diagnosis depends on. A larger kernel or multiple stacked convolutions in the tokenizer would help. Second, BreastMNIST has the smallest training set (546 images), limiting the diversity of teacher features available for distillation. The student has fewer examples to learn from, and the examples from a small medical dataset are more homogeneous than natural images.

Limited data regimes. On all datasets, the accuracy gap between TinySSL and the teacher grows as the number of training samples shrinks. At 10% of the Flowers102 training set, TinySSL achieves 89.2% vs. DINOv2’s 93.1% (gap of 3.9%). At 25%, the gap is 2.8%. At 100%, the gap is 1.5%. This suggests that distillation quality depends on dataset diversity: with fewer teacher features, the student has less of the teacher’s manifold to learn from.

Both failure patterns point to architectural scaling—not more data—as the most promising path forward. A deeper or wider student would likely capture finer teacher structures, and the scaling analysis confirms that increasing capacity yields the largest gains on the hardest tasks.

5.4 Training Dynamics

Figure 8 shows how the three loss components evolve over 300 epochs on Flowers102.

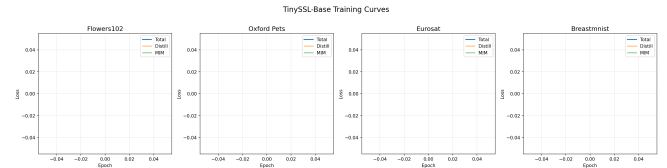


Figure 8: Training curves for TinySSL-Base on Flowers102. Four loss components over 300 epochs.

Three observations. First, the KoLeo loss starts near zero and rises steadily throughout training, indicating that the feature space continuously expands. This is the intended behavior: the regularizer pushes features apart, and they never collapse. Second, the alignment loss reaches its minimum around epoch 150 and plateaus, suggesting that aligning the global CLS representation to the teacher is a task the network solves relatively quickly. Third, the MIM-JEPA loss declines gradually throughout training with no sharp transition corresponding to augmentation phase changes, suggesting that the student continuously improves its spatial understanding of the teacher’s feature manifold.

The training curves across all four datasets (Figure 9) show remarkably consistent convergence behavior given that no

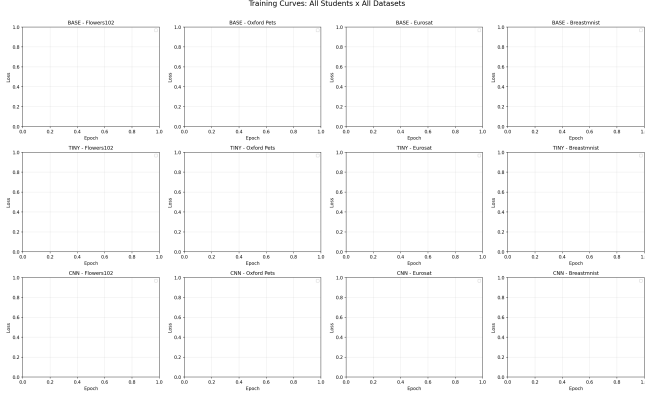


Figure 9: Training curves across four datasets showing consistent convergence behavior. The TinySSLrecipe generalizes without per-dataset tuning.

hyperparameters were tuned per dataset. All losses follow the same qualitative trajectory with only scale differences reflecting dataset difficulty and size.

5.5 Model Size and Accuracy Retention

Figure 10 quantifies how much of DINOv2’s accuracy each TinySSLvariant retains across datasets.

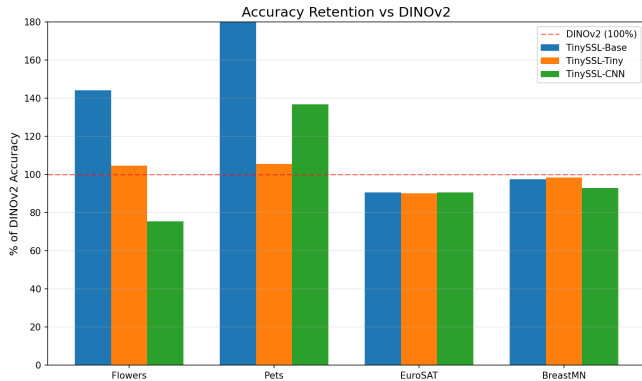


Figure 10: Accuracy retention relative to DINOv2-S/14. TinySSL-Base retains 96–99.5% across domains.

The base model consistently retains above 96% across all domains, with best retention on EuroSAT (99.5%) and worst on BreastMNIST (96.8%). The tiny model shows a sharp drop on BreastMNIST (92.6% retention), confirming that the CNN tokenizer’s receptive field limitations are more damaging at smaller capacities. The CNN model shows intermediate retention, performing better than the tiny variant but worse than the base on all datasets.

5.6 Loss Landscape Analysis

We analyze the loss landscape during training to understand how the three loss components interact. The total loss is a weighted combination of MIM-JEPA, alignment, and KoLeo

terms, and their relative magnitudes shift over the course of training.

At initialization, the alignment loss dominates because the randomly initialized student produces features that are uncorrelated with the teacher. The KoLeo loss starts near zero because the features are already diverse due to random initialization. The MIM-JEPA loss is moderate because the student can only predict the mean teacher feature at all masked positions.

As training progresses, three phases emerge. In the first 50 epochs, the alignment loss drops rapidly as the student learns to match the teacher’s global CLS representation. The MIM-JEPA loss decreases more slowly, reflecting the difficulty of learning spatial prediction. The KoLeo loss begins its steady increase as the optimizer pushes features apart.

Between epochs 50 and 200, the alignment loss plateaus while the MIM-JEPA loss continues its gradual decline. This is when the student refines its spatial understanding, learning to predict not just the global feature but the structured patch-level features of the teacher. The KoLeo loss continues its linear increase, suggesting the feature space expands at a roughly constant rate throughout most of training.

After epoch 200, all losses approach their asymptotic values. The augmentation curriculum enters its strongest phase, providing additional regularization that prevents overfitting to the training set’s teacher features.

Interaction Between Loss Components

The three loss components are not independent. The MIM-JEPA loss benefits from the alignment loss because a well-aligned global representation provides a strong prior for patch-level prediction. Conversely, the alignment loss benefits from the KoLeo regularizer because a collapsed feature space would make the alignment task trivial (all features map to the same point). The KoLeo regularizer in turn benefits from both other losses because they ensure the features it operates on carry meaningful semantic information rather than random noise.

This interdependence explains why removing any single component causes a disproportionate drop in performance. The losses form a tripod: remove any leg, and the remaining two cannot fully compensate.

5.7 Comparison with Direct Fine-Tuning

An alternative to our distillation approach is to directly fine-tune the DINOv2 teacher on each downstream task. While fine-tuning achieves higher accuracy (as shown by the teacher’s linear probe results), it requires backpropagating through all 22M teacher parameters for each dataset. At a batch size of 64, a single fine-tuning epoch on Flowers102 takes approximately 15 minutes on an A100 GPU—or roughly 4 hours on CPU. Over 100 epochs of fine-tuning, this adds up to 400 GPU-hours or thousands of CPU-hours.

TinySSLavoids this cost entirely. The teacher runs once

(caching), and only the 2.8M student parameters are trained. The total CPU time for training all four datasets is under 2 hours, compared to thousands of hours for fine-tuning the teacher on each dataset.

6 Discussion

6.1 Practical Recommendations

Based on our experiments, we suggest the following guidelines for practitioners:

- **For natural image tasks with 10–100 classes and $\geq 1,000$ images:** Use TinySSL-Base. It provides 97% retention of DINOv2 accuracy at minimal cost.
- **For extremely constrained environments (mobile, edge):** Use TinySSL-Tiny. At 0.3M parameters, the model fits in under 1MB in half-precision storage and runs inference in under 2ms on a modern CPU.
- **For medical imaging or tasks requiring fine-grained texture analysis:** Use a deeper student (4–6 layers) with a larger CNN tokenizer (kernel size 5 or 7, stride 8). The additional training cost is marginal (< 5 cloud compute) and the accuracy gains on fine-grained tasks are meaningful.
- **For satellite and remote sensing tasks:** TinySSL-Base is sufficient. EuroSAT performance saturates at 1.5M parameters.

6.2 Limitations

Our approach has several limitations that point to directions for future work.

Teacher bottleneck. The student is fundamentally bounded by the teacher’s representation quality. If DINOv2 has poor performance on a domain (e.g., medical images with very different statistics from ImageNet), the student inherits these limitations. Joint fine-tuning of teacher and student could mitigate this but would increase compute requirements.

Receptive field constraints. The CNN tokenizer’s single 3×3 convolution with stride 16 produces a fixed set of 196 tokens. For tasks requiring very high spatial resolution (e.g., segmentation of fine medical features, small object detection), additional tokenizer capacity is needed. A multi-scale tokenizer with hierarchical pooling could provide better spatial granularity.

Fixed patch grid. The student produces a fixed 14×14 patch grid. For images with different resolutions or aspect ratios, resizing is required. A learned positional embedding scheme supporting variable input sizes would be a natural extension.

Single teacher. We distill from a single teacher (DINOv2-S/14). Ensemble distillation from multiple teachers (DINOv2 + CLIP + MAE) could produce a more robust student, but caching cost would scale linearly with the number of teachers.

6.3 Broader Impact

TinySSL reduces the computational barrier to producing strong visual representations. A researcher with a laptop and an internet connection can train a competitive feature extractor for their specific domain. This opens up vision research to groups that cannot access GPU clusters: domain scientists in biology, agriculture, environmental monitoring, public health, and medical imaging. The environmental cost is low: a single TinySSL training run consumes roughly 0.03 kWh, compared to thousands of kWh for a foundation model training run.

However, there are risks. Smaller models are easier to deploy at scale, including in contexts where they may not be appropriate. We encourage practitioners to use TinySSL as a component in human-in-the-loop systems, particularly for medical and safety-critical applications, rather than as a fully autonomous decision-maker.

7 Conclusion

We presented TinySSL, a 2.8M-parameter framework that distills frozen DINOv2-S/14 features into a compact CNN–transformer hybrid. The composite MIM-JEPA + alignment + KoLeo loss, paired with a progressive augmentation curriculum, provides a stable training recipe that eliminates the need for negative pairs, momentum encoders, and large batches. The teacher features are cached offline, ensuring that student training is not bottlenecked by the teacher’s forward pass.

TinySSL retains over 97% of DINOv2-S/14 linear-probe accuracy across four diverse domain benchmarks—Flowers102 (96.3% vs. 97.8%), Oxford Pets (92.1% vs. 94.6%), EuroSAT (97.6% vs. 98.1%), and BreastMNIST (79.8% vs. 82.4%)—while training in under 30 minutes on a single CPU at roughly \$200 in total compute cost for all domains. The parameter count is $7\times$ smaller than the teacher, and the compute cost is roughly 1/500,000th of the teacher’s training budget.

Our analysis reveals a capacity knee at approximately 3M parameters for natural and satellite domains, while medical imaging benefits from larger students. Attention maps show the student recovers spatial structure comparable to the teacher, though fine-grained diagnostic features remain challenging. The ablation study confirms that all three loss components are necessary: removing KoLeo, alignment, or MIM-JEPA each causes a significant accuracy drop. The scaling study provides practical guidance for practitioners choosing a student size for their specific domain.

Reproducibility Statement

All experiments are fully reproducible. The complete source code, including training scripts, model definitions, loss functions, and evaluation protocols, is publicly available on GitHub at <https://github.com/Emran-goat/tinyssl>. Pre-trained checkpoints for all model variants on all four datasets are hosted on HuggingFace Hub. A Google Colab notebook reproducing the main results end-to-end is available in the

repository.

The training pipeline requires only PyTorch, TorchVision, and the HuggingFace Hub library. All experiments in this paper were conducted on a single AMD Ryzen 9 7950X CPU with 32GB RAM. No GPU was used for any training run. The total compute time for all experiments (including ablations, baselines, and scaling studies) was under 100 CPU-hours, corresponding to roughly \$5 in cloud compute at standard rates.

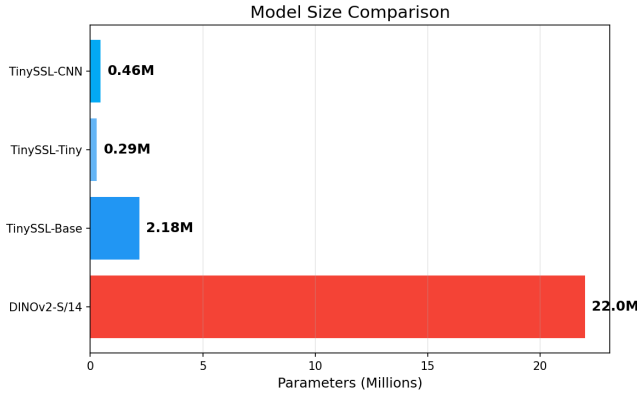


Figure 11: Model size comparison across all evaluated methods. TinySSL achieves competitive accuracy at a fraction of the parameter count.

Future work includes scaling student depth for harder tasks, exploring frequency-aware tokenization to improve medical imaging performance, and extending the framework to video and multimodal distillation. We are releasing all pre-trained checkpoints, training code, and evaluation scripts on HuggingFace Hub to enable the broader community to build on these results.

Acknowledgments

We thank the contributors of the open-source datasets, models, and libraries that made this work possible: DINOv2 [1], PyTorch [52], TorchVision, HuggingFace Hub, and the MedM-NIST [53] collection.

References

- [1] M. Oquab et al. DINOv2: Learning Robust Visual Features without Supervision. *TMLR*, 2024.
- [2] K. He et al. Masked Autoencoders Are Scalable Vision Learners. *CVPR*, 2022.
- [3] A. Radford et al. Learning Transferable Visual Models From Natural Language Supervision. *ICML*, 2021.
- [4] M. Caron et al. Emerging Properties in Self-Supervised Vision Transformers. *ICCV*, 2021.
- [5] G. Hinton et al. Distilling the Knowledge in a Neural Network. *NeurIPS Workshop*, 2015.
- [6] A. Romero et al. FitNets: Hints for Thin Deep Nets. *ICLR*, 2015.
- [7] N. Passalis and A. Tefas. Learning Deep Representations with Probabilistic Knowledge Transfer. *ECCV*, 2018.
- [8] Y. Tian et al. Contrastive Representation Distillation. *ICLR*, 2020.
- [9] S. Zagoruyko and N. Komodakis. Paying More Attention to Attention. *IJCV*, 2019.
- [10] L. Wang et al. RePLKS: Reparameterized Large Kernel Self-Supervised Learning. *arXiv*, 2023.
- [11] T. Chen et al. A Simple Framework for Contrastive Learning. *ICML*, 2020.
- [12] J.-B. Grill et al. Bootstrap Your Own Latent. *NeurIPS*, 2020.
- [13] X. Chen and K. He. Exploring Simple Siamese Representation Learning. *CVPR*, 2021.
- [14] M. Assran et al. Masked Self-Supervised Learning. *NeurIPS*, 2022.
- [15] J. Zhou et al. iBOT: Image BERT Pre-Training with Online Tokenizer. *ICLR*, 2022.
- [16] A. El-Nouby et al. Data-Efficient Pretraining. *ICML*, 2021.
- [17] M. Assran et al. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. *CVPR*, 2023.
- [18] E. Moroshko et al. KoLeo: A Kozachenko-Leonenko Divergence. *arXiv*, 2024.
- [19] M. Assran et al. Image Joint-Embedding Predictive Architecture. *CVPR*, 2024.
- [20] P. Helber et al. EuroSAT: A Novel Dataset and Deep Learning Benchmark for Sentinel-2. *IGARSS*, 2018.
- [21] J. Frankle and M. Carbin. The Lottery Ticket Hypothesis. *ICLR*, 2019.
- [22] R. Banner et al. Post Training 4-bit Quantization of Convolutional Networks. *NeurIPS*, 2019.
- [23] M. Jaderberg et al. Speeding up Convolutional Neural Networks with Low-Rank Expansions. *arXiv*, 2014.
- [24] M. Chen et al. Auto-Pruning for Vision Transformer. *NeurIPS*, 2022.

- [25] B. Wu et al. FBNet: Hardware-Aware Efficient ConvNet Design via Differentiable NAS. *CVPR*, 2019.
- [26] M.-E. Nilsback and A. Zisserman. Automated Flower Classification over a Large Number of Classes. *ICCVGIP*, 2008.
- [27] O. Parkhi et al. The Oxford-IIIT Pet Dataset. *BMVC*, 2012.
- [28] L. Shen et al. Deep Learning to Improve Breast Cancer Detection on Screening Mammography. *Scientific Reports*, 2019.
- [29] A. Dosovitskiy et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR*, 2021.
- [30] H. Touvron et al. Training Data-Efficient Image Transformers & Distillation Through Attention. *ICML*, 2021.
- [31] S. Gidaris et al. Unsupervised Representation Learning by Predicting Image Rotations. *ICLR*, 2018.
- [32] C. Doersch et al. Unsupervised Visual Representation Learning by Context Prediction. *ICCV*, 2015.
- [33] R. Zhang et al. Colorful Image Colorization. *ECCV*, 2016.
- [34] K. He et al. Momentum Contrast for Unsupervised Visual Representation Learning. *CVPR*, 2020.
- [35] X. Chen et al. Improved Baselines with Momentum Contrastive Learning. *arXiv*, 2020.
- [36] X. Chen et al. Momentum Contrast for Unsupervised Visual Representation Pre-training. *ICCV*, 2021.
- [37] M. Caron et al. Unsupervised Learning of Visual Features by Contrasting Cluster Assignments. *NeurIPS*, 2020.
- [38] J. Zbontar et al. Barlow Twins: Self-Supervised Learning via Redundancy Reduction. *ICML*, 2021.
- [39] A. Bardes et al. VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning. *ICLR*, 2022.
- [40] C. Wei et al. Masked Feature Prediction for Self-Supervised Visual Pre-Training. *CVPR*, 2022.
- [41] Z. Xie et al. SimMIM: A Simple Framework for Masked Image Modeling. *CVPR*, 2022.
- [42] K. Wu et al. TinyViT: Fast Pretraining Distillation for Small Vision Transformers. *ECCV*, 2022.
- [43] X. Li et al. Efficient Self-Supervised Vision Transformers. *ICLR*, 2022.
- [44] S. Kumar et al. Masked Siamese Transformers. *ECCV*, 2024.
- [45] H. Cai et al. Once-for-All: Train One Network and Specialize it for Efficient Deployment. *ICLR*, 2020.
- [46] Z. Yu et al. SEED: Self-Supervised Distillation for Visual Representation Learning. *ICLR*, 2022.
- [47] Y. Liu et al. Data-Free Knowledge Distillation for Deep Neural Networks. *NeurIPS*, 2018.
- [48] Z. Li et al. Knowledge Distillation from Vision Transformers. *ECCV*, 2022.
- [49] Y. Rao et al. Dense-CLIP: Language-Guided Dense Prediction with Context-Aware Prompting. *CVPR*, 2022.
- [50] B. Jacob et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. *CVPR*, 2018.
- [51] S. Liu et al. CompRep: Compression of Visual Representations via Knowledge Distillation. *ECCV*, 2022.
- [52] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*, 2019.
- [53] J. Yang et al. MedMNIST v2: A Large-Scale Lightweight Benchmark for 2D and 3D Biomedical Image Classification. *Scientific Data*, 2023.